

Solutions for Homework Assignment #5

Answer to Question 1.

a. Definition of the subproblems to solve: For each $i \in [0..n]$, let

$$C(i) = \text{the cost of an optimal placement for the first } i \text{ books,} \quad (*)$$

where we take $C(0) = 0$.

Solving the original problem: Return $C(n)$.

Recursive formula to compute each subproblem:

$$C(i) = \begin{cases} 0, & \text{if } i = 0 \\ \min \left\{ \max(C(j), L - \sum_{j < t \leq i} d_t) : j < i \text{ and } \sum_{j < t \leq i} d_t \leq L \right\}, & \text{otherwise} \end{cases} \quad (\dagger)$$

Justification why $(\dagger)$ is a correct formula to compute $(*)$: Let $P(i)$ be an optimal placement for books $1..i$, i.e., a feasible placement that minimizes the cost (as defined for this problem); and let $C(i)$ be the cost of $P(i)$, i.e., the maximum empty space on any shelf used in $P(i)$. Let k be the number of shelves in $P(i)$, and let j be the number of books in the the first $k - 1$ shelves in $P(i)$. So the last shelf contains books $j + 1..i$, for some $j < i$ such that $\sum_{j < t \leq i} d_t \leq L$.

We can assume, without loss of generality, that the first $k - 1$ shelves of $P(i)$ constitute an **optimal** placement $P(j)$ for books $1..j$. This follows by a cut-and-paste argument: If not, we can replace the first $k - 1$ shelves of $P(i)$ by $P(j)$, resulting in a placement for books $1..i$ with cost no greater than $B(i)$. Therefore, $C(i) = \max(C(j), L - \sum_{t=j+1}^i \ell_t)$. To find out the index j of the last book in the penultimate shelf of $P(i)$, we try out all possibilities (i.e., all $j < i$ such that $\sum_{t=j+1}^i \ell_t \leq L$) and pick one that minimizes the cost. Thus the recursive formula $(\dagger)$ correctly computes $C(i)$ as defined by $(*)$.

Pseudocode: (The code in the gray boxes is used in the retrofitted algorithm, part (b).)

```

1  C(0) := 0
2  for i := 1 to n do
3      x := di
4      j := i - 1
5      C(i) := max(C(j), L - x) ; pre(i) := j
6      while j > 0 and x + dj ≤ L do
7          x := x + dj
8          j := j - 1
9      if C(i) > max(C(j), L - x) then C(i) := max(C(j), L - x) ; pre(i) := j
10 return C(n)
```

Running time: The while loop is executed at most n times, so the running time of this algorithm is $O(n^2)$.

b. To find the actual optimal placement we also compute, for each $i \in [1..n]$, $pre(i)$ to be the index of the last book in the penultimate shelf of the optimal layout for books $1..i$, taking the penultimate shelf before shelf 1 to be the fictitious “shelf” 0. This is done by the pseudocode highlighted in gray. Then the optimal placement, in the reverse order, is $n, pre(n), pre(pre(n)), \dots, 0$. So, to compute the optimal placement P (in the proper order) in the retrofitted algorithm we replace line 10 by the following piece of code:

```

P := list containing only n
i := n
while i ≠ 0 do
    i := pre(i)
    prepend i to P
return P

```

Computing P in the above loop takes $O(n)$ time, so the running time of the retrofitted algorithm is $O(n^2) + O(n) = O(n^2)$.

Answer to Question 2.

Definition of the subproblems that our dynamic programming algorithm will solve: For each i , $0 \leq i \leq n$,

$$D(i) = \text{the number of legal divisions of } x[1..i] \quad (*)$$

Recursive formula to compute each subproblem:

$$D(i) = \begin{cases} 1, & \text{if } i = 0 \\ \sum_{0 \leq j < i} D(j) \cdot \text{DICT}(x, j+1, i), & \text{if } i > 0 \end{cases} \quad (\dagger)$$

Justification why $(\dagger)$ is a correct formula to compute $(*)$: For the base case $i = 0$, $x[1..0]$ is the empty string, which has a single legal division (vacuously, every string in the empty sequence is an English word; note that this is not saying that the empty string is an English word — it is not!). Therefore, $D(0) = 1$, as wanted.

For $i > 0$, consider the non-empty string $x[1..i]$. For each j such that $0 \leq j < i$, if $x[j+1..i]$ is a word (i.e., $\text{DICT}(x, j+1, i)$ returns 1) then each of the legal divisions of $x[1..j]$, followed by $x[j+1..i]$ is a distinct legal division of $x[1..i]$. The legal divisions that arise in this way for different values of j , $0 \leq j < i$, are distinct because they differ on the last string. Hence

$$D(i) \geq \sum_{0 \leq j < i} D(j) \cdot \text{DICT}(x, j+1, i). \quad (1)$$

On the other hand, any legal division of $x[1..i]$ consists of a legal division of $x[1..j]$, for some j such that $0 \leq j < i$, followed by $x[j+1..i]$, which must be an English word (otherwise the division is not legal), i.e., $\text{DICT}(x, j+1, i)$ returns 1. Hence

$$D(i) \leq \sum_{0 \leq j < i} D(j) \cdot \text{DICT}(x, j+1, i). \quad (2)$$

By (1) and (2), we have that, for $i > 0$, $D(i) = \sum_{0 \leq j < i} D(j) \cdot \text{DICT}(x, j+1, i)$, as wanted.

Solving the original problem: By the definition of the subproblems $(*)$, the required return value is $D(n)$.

Pseudocode:

```

LEGALDIVISIONS(x)
1  n := length(x)
2  D(0) := 1
3  for i := 1 to n do
4      D(i) := 0
5      for j := 0 to i-1 do
6          D(i) := D(i) + D(j) · DICT(x, j+1, i)
7  return D(n)

```

Running time: The running time of the algorithm is dominated by the doubly-nested loop in lines 3-6, which takes $\Theta(n^2)$ time.

Answer to Question 3.

Definition of the subproblems to solve: For each i , $1 \leq i \leq n$,

$V_T(i)$ = maximum profit for days $1, 2, \dots, i$ of a schedule that ends in Toronto (*)

$V_M(i)$ = maximum profit for days $1, 2, \dots, i$ of a schedule that ends in Melbourne

(We are not interested in schedules ending on a travel day since, by assumption, the profit attainable each day on either city is non-negative.)

Solving the original problem: $\max(V_T(n), V_M(n))$.

Recursive formula to compute each subproblem: For each i , $1 \leq i \leq n$,

$$\begin{aligned} V_T(i) &= \begin{cases} T[1], & \text{if } i = 1 \\ T[1] + T[2], & \text{if } i = 2 \\ \max(V_T(i-1) + T[i], V_M(i-2) + T[i]), & \text{if } i > 2 \end{cases} \\ V_M(i) &= \begin{cases} M[1], & \text{if } i = 1 \\ M[1] + M[2], & \text{if } i = 2 \\ \max(V_M(i-1) + M[i], V_T(i-2) + M[i]), & \text{if } i > 2 \end{cases} \end{aligned} \quad (\dagger)$$

Justification why $(\dagger)$ is a correct formula to compute $(*)$: The base cases $i \in \{1, 2\}$ are obvious. For the general case $i > 1$, consider an optimal schedule $S_T(i)$ for days $1, 2, \dots, i$ that ends in Toronto (the justification for a schedule $S_M(i)$ for days $1, 2, \dots, i$ that ends in Melbourne is similar). There are two cases:

CASE 1. In day $i-1$, Emon was in Toronto. In this case, by a cut-and-paste argument, $S_T(i)$ consists of an optimal schedule $S_T(i-1)$ for days $1, 2, \dots, i-1$ that ends in Toronto, resulting in profit $V_T(i-1)$, and Emon working on the Toronto project on day i , resulting in an additional profit of $T[i]$. So, in this case, $V_T(i) = V_T(i-1) + T[i]$.

CASE 2. In day $i-1$, Emon was traveling from Melbourne to Toronto. In this case, by a cut-and-paste argument, $S_T(i)$ consists of an optimal schedule $S_M(i-2)$ for days $1, 2, \dots, i-2$ that ends in Melbourne, resulting in profit $V_M(i-2)$, then Emon traveling from Melbourne to Toronto on day $i-1$, resulting in no profit, and then working on the Toronto project on day i , resulting in an additional profit of $T[i]$. So, in this case, $V_T(i) = V_M(i-2) + T[i]$.

Combining the two cases we get $\max(V_T(i-1) + T[i], V_M(i-2) + T[i])$.

Pseudocode:

```

MAXVALUE( $T[1..n], M[1..n]$ )
1   $V_T(1) := T[1]; V_T(2) := T[1] + T[2]$ 
2   $V_M(1) := M[1]; V_M(2) := M[1] + M[2]$ 
3  for  $i := 3$  to  $n$  do
4       $V_T(i) := \max(V_T(i-1) + T[i], V_M(i-2) + T[i])$ 
5       $V_M(i) := \max(V_M(i-1) + M[i], V_T(i-2) + M[i])$ 
6  return  $\max(V_T(n), V_M(n))$ 

```

Running time: The running time of the algorithm is dominated by the loop in lines 3-5, which takes $O(n)$ time.